Practice Similar Questions using OOP, Exception Handling(Validations), File Handling, JDBC (Performing only CRUD Using DB), Lambda expressions, Predicates, Stream API, Records, Enum and LocalDate methods.

Package Structure:

com.cdac.test: Tester.Java- main()

com.cdac.utils: Helper class with static methods and Collection

Public class Helper{

Private List<T> list;

Public Helper(){

this.list= new ArrayList<Your class>();

}

//creating a object of this class will give u a collection to work on.

//add helper methods in this class only if required. If the operations are done in tester using stream API, no need of this class.

//You may create a populate() method for dummy collection so, you can easily test your code. → in main: List<> list= Util.populate();

com.cdac.io/com.cdac.db: JDBC or File Handling (Use OOS and OIS) Code

Best of luck for the Lab Exam!!!

For B1:

Student Result Management System:

Create a console-based Java program to manage and analyze student examination results. Each Result record should include:

- Student ID (unique, auto-generated)

- Student Name

- Subject Name

- Marks Obtained (int, out of 100)

- Grade (use Enum: DISTINCTION, FIRST_CLASS, SECOND_CLASS, PASS, FAIL)

- Exam Date (use LocalDate)

Program should provide the following functionalities:

1. Add a new Result entry: Allow the user to add a student result. Student ID should be auto-generated. Grade must be assigned automatically based on marks (>=75: DISTINCTION, >=60: FIRST_CLASS, >=50: SECOND_CLASS, >=40: PASS, else FAIL). Store in memory.

2. Display results for a specific Grade: Use a Predicate<Result> (named byGrade) to filter results by a given Grade Enum value and display using Stream API.

3. Calculate average marks by Subject: Use a Predicate<Result> (named bySubject) to filter results for a given subject using Stream API.

4. Find the Top Scorer: Use the Stream API to find and display the student with the highest marks. If no results exist, display an appropriate message.

5. Display all results sorted by Marks (Descending): Use Stream API with Comparator to sort and display all results in descending order of marks obtained.

Note:

· Use appropriate Java collections like List<Result> to store result entries in memory.

· Use Enum Grade and assign grade automatically during result creation.

· Use Predicate<Result> to encapsulate reusable filter conditions (byGrade, bySubject).

· Use Java 8 Predicate, Stream API, Optional, Comparator, and Lambda expressions where applicable.

· Follow Java conventions when naming packages, classes, methods, variables and modularity

· Implement custom exception handling for invalid inputs such as marks out of range or empty fields.

For B2:

Banking Account Management System:

Create a console-based Java program to manage bank accounts using inheritance and polymorphism. Implement a base abstract class Account and derive SavingsAccount and CurrentAccount.

Each Account should include:

- Account Number (unique, auto-generated)

- Account Holder Name

- Account Type (use Enum: SAVINGS, CURRENT)

- Balance (double)

- Date of Opening (use LocalDate)

Program should provide the following functionalities:

1. **Create Account:** Allow the user to create a SavingsAccount or CurrentAccount. SavingsAccount must have a minimum balance of 500.0; CurrentAccount must have an overdraft limit of 10000.0. Throw InvalidAccountException for invalid inputs.

2. **Deposit Amount:** Perform deposit on a given account; override deposit() in each subclass. Display updated balance after deposit.

3. **Withdraw Amount:** Perform withdrawal with polymorphic behavior - SavingsAccount must maintain minimum balance; CurrentAccount allows overdraft up to its limit. Throw InsufficientFundsException if withdrawal fails.

4. **Display All Accounts**: Display all accounts sorted by balance in descending order using Stream API and Comparator.

5. **Display Account Summary by Type**: Use Stream API and Enum to group accounts by type (SAVINGS / CURRENT) and display the count and total balance for each type.

Note:

· Create an abstract class Account with common fields and abstract methods deposit(double amount) and withdraw(double amount).

· SavingsAccount and CurrentAccount must extend Account and override deposit() and withdraw() methods.

· Use polymorphism - store all accounts in List<Account> and invoke methods via base class reference.

· Use Enum AccountType for type classification.

- Follow Java conventions when naming packages, classes, methods, variables and modular approach.
- Use custom exception handling for invalid inputs or insufficient funds.

GitHub Link has been updated.

Refer to Github link for learning material …
github.com/sohammehtaa/PGCP-AC

Practice Test Results have been shared via email. Generate similar tests using Claude.ai by attaching the module teaching guidelines & practice for exam.

Regarding any queries you may contact on linkedin:
https://www.linkedin.com/in/sohammehta20
or
mail: sohamm@cdac.in

Wishing you all the best in your future endeavors…